Rapport de stage

Fred Belanger
felix.belanger@universite-paris-saclay.fr

16 Avril 2026

Abstract

On a fait des betises sur les problemes de typage

Contents

1 Introduction . . . 1
2 Présentation des types ensemblistes . . . 2
3 Génération d'un témoin . . . 3
3.1 Définition d'un témoin . . . 3
3.2 Définiton de $w : t$ . . . 3
4 Présentation de l'algorithme . . . 3
5 En Pratique . . . 4
5.1 Cas de bases . . . 4
5.1.1 Int . . . 4
5.1.2 Enum . . . 4
5.1.3 Arrow . . . 5
5.2 Constructeurs . . . 5
5.2.1 Tuple . . . 5
5.2.2 Tag . . . 5
5.2.3 records . . . 5
6 Preuve de sûreté . . . 6
6.1 Preuve de la terminaison de $t \rightsquigarrow w$ . . . 6
6.2 Preuve de la justesse de l'algorithme . . . 7
Bibliography . . . 8

1 Introduction

Le but du stage a été de créer une fonction permettant, pour tout type non-vide, de trouver un habitant de ce type, appelé témoin. Ce témoin permet d'afficher un exemple lors de la diffusion d'un message d'erreur pour mauvais typage.

Exemple On a une fonction $f : \mathsf{Int} \to \mathsf{Int}$ et une variable $x : \mathsf{Int} \vee \mathrm{Bool}$. Alors l'application $f(x)$ est mal typée, car $\mathsf{Int} \vee \mathrm{Bool}$ n'est pas un sous-type de Int. On cherche donc un témoin de $(\mathsf{Int} \vee \mathrm{Bool}) \setminus \mathsf{Int} = \mathrm{Bool}$, ce qui peut rendre True ou False. Pour des types plus complexes, on veut automatiser le processus, ce qui amène à l'algorithme "Witness" expliqué ici.

2 Présentation des types ensemblistes

Les types utilisés dans ce rapport ont été introduits par M. Laurent, K. Nguyen et G. Castagna dans “A Gentle Introduction to Semantic Subtyping” (Castagna 2005) et “Implementing Set-Theoretic Types” (Laurent and Nguyen 2025) comme des types ensemblistes. On les définit comme :

$$t ::= b \mid \alpha \mid t \to t \mid t \times t \mid t \vee t \mid t \wedge t \mid \neg t \mid \mathbb{0} \mid \mathbb{1} \tag{1}$$

Où $b \in \mathcal{B}$ sont les cas de bases (en particuliers les constantes $c \in \mathcal{C}$), $\alpha \in \mathcal{V}$ les variables de types, $\mathbb{0}$ le type vide et $\mathbb{1}$ le supertype de tout les types. $\mathcal{B}$ est composé de Int (l'ensemble des entiers relatifs) et Enum (l'ensemble des types énumérés, par exemple bool).

Exemple : $t = (\mathsf{Int}, (\text{True} \to \text{False})) \vee 42$ est un type correct dans notre syntaxe.

On définit $[\![t]\!]$ comme l'ensemble des valeurs contenues dans t, ce qui nous permet de définir aussi la relation de sous-typage $t_1 \leqslant t_2$ qui est vrai si et seulement si $[\![t_1]\!] \in [\![t_2]\!]$. On peut noter qu'il existe des cas, comme Bool et Int, où Bool $\not\leqslant \mathsf{Int}$ et $\mathsf{Int} \not\leqslant$ Bool. On a aussi que $\forall t, \mathbb{0} \leqslant t \leqslant \mathbb{1}$. On peut enfin définir l'égalité sémantique $t_1 \simeq t_2$ comme étant vrai si et seulement si $t_1 \leqslant t_2$ et $t_2 \leqslant t_1$.

Comme ces types sont définis co-inductivement, ils peuvent être définis comme des arbres infinis réguliers et contractifs.

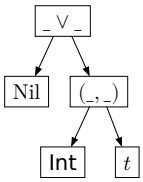
Definition 2.1

Un arbre est dit **régulier** si il a un nom fini de sous-termes possibles.

Definition 2.2

Un arbre est dit **contractif** si une branche infinie passe forcément par un constructeur (ici, $\to$ et $\times$).

Exemple $t = \text{Nil} \vee (\mathsf{Int}, t)$ sous forme d'arbre:



On peut aussi représenter celà sous la forme d'un graphe cyclique en remplaçant la feuille t par une flèche vers _ ∨ _. Ceci nous permet d'avoir une définition de type qui n'a pas besoin de définitions explicites pour la récursion.

Enfin, on considère les types infinis comme vides, car ils sont inconstructibles en pratique (ex : $t = (\mathsf{Int}, t) \leqslant \mathbb{0}$).

Dans un premier temps, nous allons ignorer les variables de types pour la génération du témoin.

3 Génération d'un témoin

3.1 Définition d'un témoin

Un témoin est un habitant de notre type, c'est à dire une valeur constante apartenant à $[\![t]\!]$. Notre but est, pour chaque cas de base, de trouver un habitant de ce cas, et pour chaque constructeur, de chercher récurisvement un témoin pour hcacune de ses parties.

Contrairement à ce que voudrait la logique, les atomes de la forme $\bigwedge(t_1 \to t_2) \wedge \bigwedge \neg(t_1 \to t_2)$ sont ici considérés comme des cas de base, car toutes les fonctions sont habitées, et connaitre uniquement un cas d'utilisation serait complexe et improductif. on considère donc que $A ::= \bigwedge(t_1 \to t_2) \wedge \bigwedge \neg(t_1 \to t_2)$ est un cas de base.

On propose donc comme type témoin :

$$w ::= c \mid A \mid w \times w \tag{2}$$

En pratique, on a $c ::= i \in [\![\mathsf{Int}]\!] \mid e \in [\![\text{Enum}]\!]$, ainsi que des extensions pour les tags, les n-uplets et les records, ce qui donne :

$$w ::= i \in [\![\mathsf{Int}]\!] \mid e \in [\![\text{Enum}]\!] \mid A \mid w \times w \times ... \times w \mid \text{tag}(w) \mid \{l_i : w ... l_n : w\} \tag{3}$$

Les tags, les n-uplets et les records ne sont en réalité que des cas particuliers des tuples, ils ne seront donc pas traités sur le plan théorique.

Exemple : $w = (42, (\text{True}, \text{False}))$ est un témoin correct dans notre syntaxe

3.2 Définiton de $w : t$

On peut donc définir un prédicat $w : t$ qui est vrai si et seulement si w est un témoin de t, avec les règles suivantes :

$$\frac{c \in [\![b]\!]}{c : b}\,\text{Base}_w \qquad \frac{w \leqslant A}{\vdash w : A}\,\underset{w}{\to}$$

$$\frac{\vdash w_1 : t_1 \qquad \vdash w_2 : t_2}{\vdash (w_1 \times w_2) : (t_1 \times t_2)}\,\times_w \qquad \frac{\vdash w : t' \qquad t' \leqslant t}{\vdash w : t}\,\text{Sous-type}_w$$

Exemple On veut montrer que $w = (3, (\text{True}, \mathsf{Int} \to \mathsf{Int}))$ est bien un témoin de $t = (\mathsf{Int}, (\text{Bool}, \mathbb{0} \to \mathbb{1}))$:

$$\frac{\dfrac{3 \in [\![\mathsf{Int}]\!]}{3 : \mathsf{Int}}\,\text{Base}_w \qquad \dfrac{\dfrac{\text{True} \in [\![\text{Bool}]\!]}{\text{True} : \text{Bool}}\,\text{Base}_w \qquad \dfrac{\mathsf{Int} \to \mathsf{Int} \leqslant \mathbb{0} \to \mathbb{1}}{\vdash \mathsf{Int} \to \mathsf{Int} : \mathbb{0} \to \mathbb{1}}\,\underset{w}{\to}}{(\text{True}, \mathsf{Int} \to \mathsf{Int}) : (\text{Bool}, \mathbb{0} \to \mathbb{1})}\,\times_w}{(3, (\text{True}, \mathsf{Int} \to \mathsf{Int})) : (\mathsf{Int}, (\text{Bool}, \mathbb{0} \to \mathbb{1}))}\,\times_w$$

4 Présentation de l'algorithme

On peut maintenant décrire le fonctionnement de l'algorithme qui pour chaque type t associe un témoin w (qu'on notera $t \rightsquigarrow w$ et appellera Witness(t)) qui a les règles suivantes :

$$\frac{w = c \leqslant b}{\Delta \vdash_s b \rightsquigarrow w}\,\text{Base}_t \qquad \frac{w \leqslant t_1 \to t_2}{\Delta \vdash_s t_1 \to t_2 \rightsquigarrow w}\,\underset{t}{\to} \qquad \frac{\Delta \vdash_m t_1 \rightsquigarrow w_1 \qquad \Delta \vdash_m t_2 \rightsquigarrow w_2}{\Delta \vdash_s t_1 \times t_2 \rightsquigarrow w_1 \times w_2}\,\times_t$$

$$\frac{\Delta \vdash_m t_1 \rightsquigarrow w}{\Delta \vdash_s t_1 \vee t_2 \rightsquigarrow w}\,\vee_{1_t} \qquad \frac{\Delta \vdash_m t_2 \rightsquigarrow w}{\Delta \vdash_s t_1 \vee t_2 \rightsquigarrow w}\,\vee_{2_t} \qquad \frac{\Delta \cup \{t\} \vdash_s t \rightsquigarrow w}{\Delta \vdash_m t \rightsquigarrow w}\,t \notin \Delta$$

Ici, Δ est l'ensemble des types déjà visité par l'algorithme. Ainsi, si l'on retombe sur le même type, on sait que celui-ci est infini, donc vide (comme expliqué ici **RAJOUTER LIEN**). On interdit donc de repasser plusieurs par un type déjà visité avec la règle $t \notin \Delta$.

Exemple : On a $t = (\mathsf{Int}, (\mathsf{Int} \to \text{Bool}) \vee (\text{Bool} \to \mathsf{Int}))$

Cela nous donne l'arbre :

$$\frac{\dfrac{w_1 = 42 \leqslant \mathsf{Int}}{\Delta = \{\mathsf{Int}\} \vdash_s \mathsf{Int} \rightsquigarrow w_1}\,\text{Base}_t}{\vdash_m \mathsf{Int} \rightsquigarrow w_1}\,\mathsf{Int} \notin \Delta \qquad \frac{\dfrac{\dfrac{w_2 = \mathsf{Int} \to \mathbb{0} \leqslant \mathsf{Int} \to \text{Bool}}{\Delta \cup \{\mathsf{Int} \to \text{Bool}\} \vdash_s \mathsf{Int} \to \text{Bool} \rightsquigarrow w_2 = \mathsf{Int} \to \mathbb{0}}\,\underset{t}{\to}}{\Delta \vdash_m \mathsf{Int} \to \text{Bool} \rightsquigarrow w_2}\,\mathsf{Int} \to \text{Bool} \notin \Delta}{\Delta = \{(\mathsf{Int} \to \text{Bool}) \vee \text{Nil}))\} \vdash_s (\mathsf{Int} \to \text{Bool}) \vee \text{Nil}) \rightsquigarrow w_2}\,\vee_{1_t}}{\vdash_m t = (\mathsf{Int} \to \text{Bool}) \vee \text{Nil}) \rightsquigarrow w_2}\,t \notin \Delta$$
$$\frac{\vdash_m \mathsf{Int} \rightsquigarrow w_1 \qquad \vdash_m t = (\mathsf{Int} \to \text{Bool}) \vee \text{Nil}) \rightsquigarrow w_2}{\vdash_s (\mathsf{Int}, (\mathsf{Int} \to \text{Bool}) \vee \text{Nil}) \rightsquigarrow (w_1, w_2)}\,\times_t$$

On peut ainsi vérifier que $w = (42, \mathsf{Int} \to \mathbb{0})$ est bien un témoin de $t = (\mathsf{Int}, (\mathsf{Int} \to \text{Bool}) \vee (\text{Bool} \to \mathsf{Int}))$:

$$\frac{\dfrac{42 \in [\![\mathsf{Int}]\!]}{42 : \mathsf{Int}}\,\text{Base}_w \qquad \dfrac{\dfrac{\mathsf{Int} \to \mathbb{0} \leqslant (\mathsf{Int} \to \text{Bool})}{\vdash \mathsf{Int} \to \mathbb{0} : (\mathsf{Int} \to \text{Bool})}\,\underset{w}{\to} \qquad (\mathsf{Int} \to \text{Bool}) \leqslant (\mathsf{Int} \to \text{Bool}) \vee \text{Nil}}{\vdash \mathsf{Int} \to \mathbb{0} : (\mathsf{Int} \to \text{Bool}) \vee \text{Nil}}\,\text{Sous-type}_w}{\vdash (42, \mathsf{Int} \to \mathbb{0}) : (\mathsf{Int}, (\mathsf{Int} \to \text{Bool}) \vee \text{Nil})}\,\times_w$$

5 En Pratique

L'algorithme va essayer de trouver un témoin dans chacune des 6 grandes catégories (Int, Enum, Arrows, Tag, Tuple, Record) dans cet ordre, et s'arrêter dès qu'un est trouvé. Si aucun témoin n'est trouvé, on considère le type comme vide et on lève une exception.

5.1 Cas de bases

5.1.1 Int

Les entiers sont représentés comme des intervalles. Il existe 3 cas possibles :

- $t =]-\infty; +\infty[$: l'algorithme retourne 42.
- $t =]-\infty; i]$ ou $t = [i; +\infty[$: retourne i.
- $t = [i_1; i_2]$: retourne i_1.

5.1.2 Enum

Enum contient tout les types énumérés, incluant notamment les booléens (et les Strings ?). Il peuvent être définis de 2 manières :

- comme une union des constantes appartenant à t : on renvoie la première
- comme une union des constantes appartenant à $\neg t$: On retourne une constante de Enum de taille n, n n'étant la taille d'aucune des constantes de $\neg t$

Exemples
1. True | False donnera True.
2. Enum $\setminus$ ($a \vee$ ba $\vee$ toto) donnera abc.

5.1.3 Arrow

Comme dit plus haut, on considère les atomes de la forme $\bigwedge(t_1 \to t_2) \wedge \bigwedge \neg(t_1 \to t_2)$ comme des cas de base. On va donc prendre tout les atomes positifs de t, et y rajouter un par un les atomes négatifs de t. Si au moment d'un ajout, l'intersection des atomes positifs et négatifs donne un sous-type de t, c'est notre témoin.

5.2 Constructeurs

5.2.1 Tuple

Les n-uplets peuvent, comme les Enum, être défini comme les éléments présents dans t ou comme les éléments présents dans $\neg t$.

- Si on a la liste des éléments présents dans t, on choisit l'élément e de plus petite arité dont aucun élément n'est vide, et on renvoie un n-uplet de la même taille et dont chaque élément est le témoin du type à la même position dans e.
- Si on a la liste des éléments présents dans $\neg t$, on trouve le plus petit entier n tel qu'il n'y ai aucun n-uplet de cette arité dans la liste, et on en renvoie un témoin (usuellement le n-uplet (0,1,...,n))

Exemples
1. $(\mathsf{Int}, \text{Bool})$ donnera $(42, \text{True})$
2. Tuple $\setminus$ $((\mathsf{Int},) \vee (\mathsf{Int}, \text{Bool}))$ donnera $(0, 1, 2)$

5.2.2 Tag

Les tags sont des tuples avec un type Tag (équivalent à un type string) en premier membre et un type t en deuxième. Ils servent à marquer certains types pour ne pas être confondus (ex : foo(Int)). Ils sont traités de la même manière que les tuples de taille 2, à l'exception que l'on n'opère pas de récursion sur le premier membre.

Exemples
1. foo(Int) donnera foo(42).
2. Tag $\setminus$ foo(Int) donnera a(16).

5.2.3 records

Les atomes de records sont de la forme

$$r ::= \{\text{bindings} : (\text{Label} \times f)\ \text{map}; \text{tail} : f; \text{exists} : (\text{Label Set} \times f)\ \text{Map}\} \tag{4}$$

- Ici, le type f est une extension de notre type t qui inclus les types optionnels (ex : $f = \mathsf{Int}?$). On le défini comme $f \in \mathbb{1} \cup \bot$, $\bot$ dénotant le type absent.
- Le bindings est un champ où l'on place toutes les variables nommées de notre record.

Exemple $\{x : \mathsf{Int}; y : \mathsf{Int}?; \text{predicat} : \text{Bool}\}$

- La tail f' indique si notre record est ouvert ou fermé (donc s'il peut avoir des variables supplémentaires non nommées).

Exemple $\{x : \mathsf{Int};\ \ y : \mathsf{Int};;\ \ \mathsf{Int}?\}$ pour un système de coordonnées à au moins 2 dimensions.

- Pour chaque paire $(L_i : e_i)$ de exists, L_i indique les noms que la variables ne peut PAS prendre et $e_i \wedge f'$ indique son type.

Exemple $\{x : \mathsf{Int}; y : \mathsf{Int};; \text{Any?};; \{w; z\} : \text{Bool?}\}$ pour indiquer que les variables w et z ne peuvent pas avoir le type Bool si elles existent.

Pour générer un témoin, on va trouver un atome non vide de notre record, et opéré comme suit :

- Pour chaque paire $(l_i : f_i)$ de bindings, si f_i est requis, alors on cherche récursivement un témoin w_i de f_i et ajouter $(l_i : w_i)$ au bindings de notre témoin w.

Exemple $\{x : \mathsf{Int}; y : \mathsf{Int}?; \text{predicat} : \text{Bool}\}$ donnera $\{x : 42; \text{predicat} : \text{True}\}$.

- Si la tail f' est requise, on l'ajoute dans la tail de w.

Exemples
1. $\{x : \mathsf{Int};; \text{Bool}\}$ donnera $\{x : 42;; \text{Bool}\}$.
2. $\{x : \mathsf{Int};; \mathsf{Int}?\}$ donnera $\{x : 42\}$.

- Pour chaque paire $(L_i : e_i)$ de exists, si $e_i \wedge f$ est requis, alors on crée un label $l'_i \notin L_i$, on cherche récursivement un témoin w'_i de $e_i \wedge f$ et on ajoute $(l'_i : w'_i)$ **au bindings**.

Exemple $\{x : \mathsf{Int}; y : \mathsf{Int};; \text{Any?};; \{w; z\} : \text{Bool}\}$ donnera $\{x : 42; y : 42; a : \text{True}\}$

6 Preuve de sûreté

On veut montrer que le socle théorique de notre algorithme est fondé, nous allons donc montrer la terminaison de notre programme, ainsi que la preuve théorique de sa justesse.

6.1 Preuve de la terminaison de $t \rightsquigarrow w$

> **Lemme 6.1**
>
> $\forall t, t \leqslant \mathbb{1} \times \mathbb{1} \Rightarrow \exists (t_1^i, t_2^i), t \simeq \bigvee_i (t_1^i, t_2^i)$

Proof: Si $t \leqslant \mathbb{1} \times \mathbb{1}$ alors :

$$\begin{aligned} t &= \bigvee\Big(\bigwedge_i (p_1^i, p_2^i) \wedge \bigwedge_j \neg\big(n_1^j, n_2^j\big)\Big) \\ &= \bigvee\Big(\big(\bigwedge_i (p_1^i, p_2^i)\big) \setminus \bigvee_j \big(n_1^j, n_2^j\big)\Big) \\ &= \bigvee\Big((\wedge_i p_1^i, \wedge_i p_2^i) \setminus \bigvee_j \big(n_1^j, n_2^j\big)\Big) \quad \text{par commutativité de } \wedge \end{aligned}$$

Montrons par récurrence sur $\bigvee_j \big(n_1^j, n_2^j\big)$ que t est bien une union :

On a bien $(\wedge_i p_1^i, \wedge_i p_2^i)$ qui est une union d'un seul élement.

On a maintenant $\bigvee_i (t_1^i, t_2^i)$ une union, montrons que $\bigvee_i (t_1^i, t_2^i) \setminus (n_1, n_2)$ est une union : $\bigvee_i (t_1^i, t_2^i) \setminus (n_1, n_2) = \bigvee_i ((t_1^i, t_2^i) \setminus (n_1, n_2))$

$$= \bigvee_i ((t_1^i \setminus n_1, t_2^i) \vee (t_1^i, t_2^i \setminus n_2)) \quad \text{qui est une union.}$$

Donc par récurrence, $\forall t, t \leqslant \mathbb{1} \times \mathbb{1} \Rightarrow \exists (t_1^i, t_2^i), t \simeq \bigvee_i (t_1^i, t_2^i)$. □

Théorème 6.2

Pour tout type t non-vide, $t \rightsquigarrow w$ termine.

Preuve : On peut déjà noter que pour tout type t, Δ a une taille maximale finie, car il est au maximum de la taille l'ensemble de tout les sous-types de t (qui est fini car t est régulier selon la Definition 2.1).

A chaque appel récursif, Δ grandit, nous ne pouvons donc pas avoir d'appel récursif infinis. On peut donc ignorer ce cas là par la suite.

En peut aussi dire qu'en pratique, nos types sont stockés sous la forme de DNF, on a donc :

$$t = \bigvee\left(\bigwedge b\right)\bigvee\left(\bigwedge t_1 \to t_2\right)\bigvee\left(\bigwedge(t_1 \times t_2)\right) \tag{5}$$

Grâce au Lemme 6.1, on peut réécrire t comme :

$$t = \bigvee\left(\bigwedge b\right)\bigvee\left(\bigwedge t_1 \to t_2\right)\bigvee(t_1 \times t_2) \tag{6}$$

Il suffit donc de montrer que $t \rightsquigarrow w$ termine si $t = \bigwedge b$, $t = \bigwedge t_1 \to t_2$, $t = t_1 \times t_2$ et $t = t_1 \vee t_2$

Si $t = \bigwedge b$, alors il suffit de prendre un atome de $\bigwedge b$, ce qui se fait en temps fini.

Si $t = \bigwedge t_1 \to t_2$, alors $\bigwedge t_1 \to t_2$ est une solution, donc cela se fait aussi en temps fini.

Supposons maintenant que $t_1 \rightsquigarrow w_1$ et $t_2 \rightsquigarrow w_2$ terminent.

Si $t = t_1 \times t_2$, alors par $\times_t$, $t_1 \times t_2 \rightsquigarrow w_1 \times w_2$ termine si $t_1 \rightsquigarrow w_1$ et $t_2 \rightsquigarrow w_2$ terminent, ce qui est le cas par hypothèse. Donc $t_1 \times t_2 \rightsquigarrow w_1 \times w_2$ termine.

Si $t = t_1 \vee t_2$, alors :
- Par $\vee_{1_t}$, $t_1 \vee t_2 \rightsquigarrow w$ termine si $t_1 \rightsquigarrow w$ termine, ce qui est le cas par hypothèse
- Par $\vee_{2_t}$, $t_1 \vee t_2 \rightsquigarrow w$ termine si $t_2 \rightsquigarrow w$ termine, ce qui est le cas par hypothèse.

Donc si $t = t_1 \vee t_2$, alors $t \rightsquigarrow w$ termine.

Donc par induction, pour tout type t non-vide, $t \rightsquigarrow w$ termine. □

6.2 Preuve de la justesse de l'algorithme

Théorème 6.3

Pour tout type t non-vide, si $\emptyset \vdash {}_s t \rightsquigarrow w$ alors $w : t$.

Preuve : On définit $P(t, w) :=$ Si $\emptyset \vdash {}_s t \rightsquigarrow w$ alors $w : t$
On prouve par induction sur les règles d'inférences de $t \rightsquigarrow w$:

Si $\vdash {}_s b \rightsquigarrow w$, alors par Base_t, $w = c \leqslant b$, donc $w = c \in [\![b]\!]$, donc $w : b$.

Si $\vdash {}_s(\bigwedge t_1 \to t_2) \rightsquigarrow w$, alors par $\underset{t}{\to}$, $w = w_1 \to w_2 \leqslant \bigwedge t_1 \to t_2$, donc par $\underset{w}{\to}$, $w : \bigwedge t_1 \to t_2$

Supposons maintenant $P(t_1, w)$ et $P(t_2, w)$.

Montrons que $P(t_1 \times t_2, w_1 \times w_2)$:
Si $(t_1 \times t_2) \rightsquigarrow (w_1 \times w_2)$, alors par $\times_t$, $t_1 \rightsquigarrow w_1$ et $t_2 \rightsquigarrow w_2$, donc par hypothèse, $w_1 : t_1$ et $w_2 : t_2$, donc par $\times_w$, $(w_1 \times w_2) : (t_1 \times t_2)$.

Montrons que $P(t_1 \vee t_2, w)$:
Si $t_1 \vee t_2 \rightsquigarrow w$, alors par $\vee_{1_t}$, $t_1 \rightsquigarrow w$, donc $w : t_1$, or $t_1 \leqslant t_1 \vee t_2$ donc par Sous-type$_w$, on a $w : t_1 \vee t_2$, donc si .

Donc pour tout type t non-vide, si $\emptyset \vdash_m t \rightsquigarrow w$ alors $w : t$. $\square$

Bibliography

[1] G. Castagna, "Semantic subtyping: challenges, perspectives, and open problems," no. 3701, pp. 1–20, 2005.

[2] M. Laurent and K. Nguyen, "Implementing Set-Theoretic Types," Nov. 2025, [Online]. Available: https://hal.science/hal-05369012